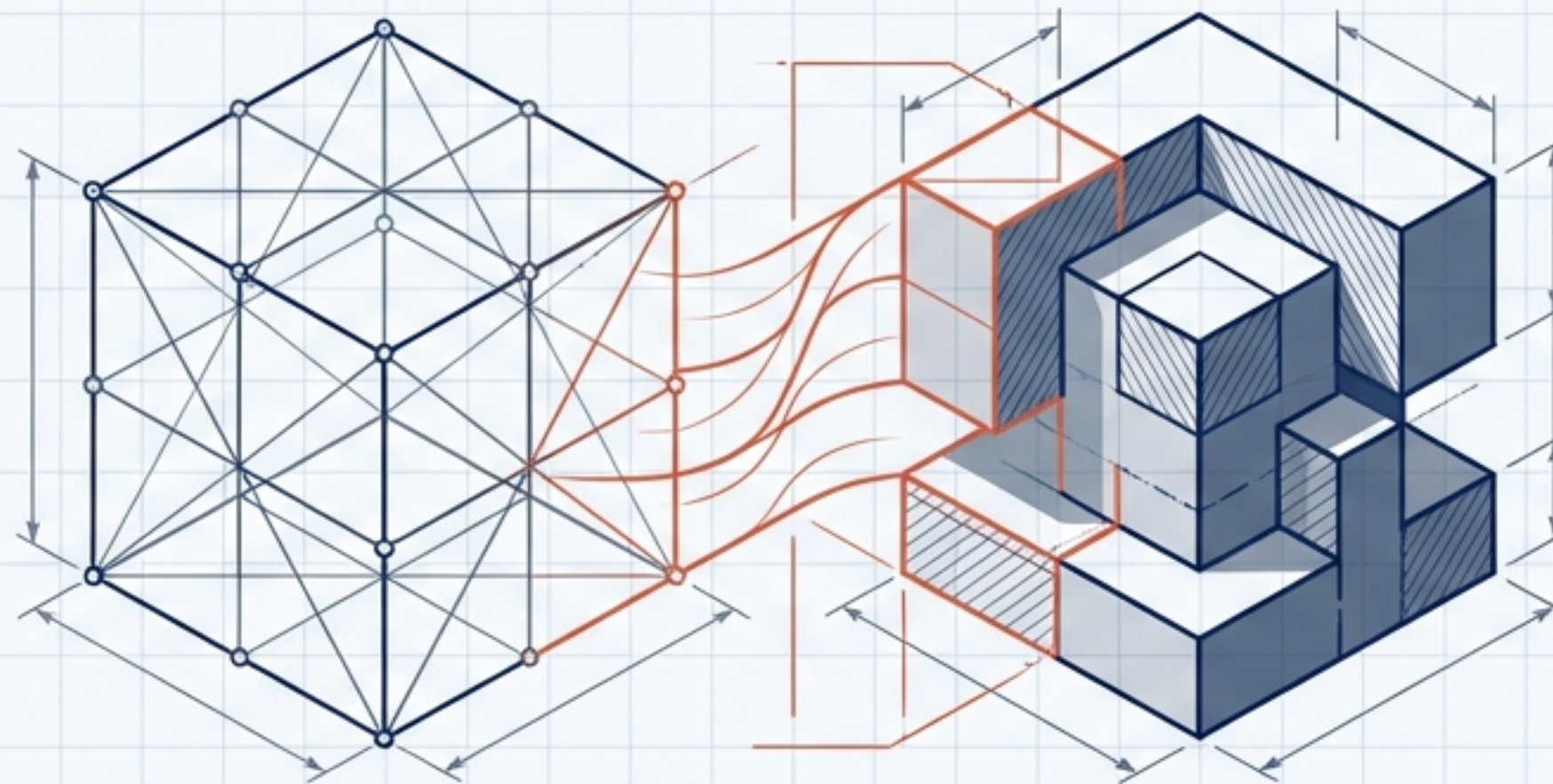


Arquitectura de Software: Dominando el Diagrama de Clases UML

De la Estructura Estática al Código Fuente



Basado en los estándares de modelado orientado a objetos.

El Mapa de la Estructura Estática

¿Qué es?

El diagrama de clases es el bloque de construcción principal del modelado orientado a objetos. No muestra el flujo de eventos, sino la estructura estática: qué objetos existen, sus atributos y quién se conecta con quién.

Niveles de Abstracción:

- 1. Modelo de Dominio (Comprensión del Negocio)
- 2. Modelo de Análisis (Refinamiento Lógico)
- 3. Modelo de Diseño (Especificación de Software)



La Anatomía de una Clase

Estudiante

nombre : String
cédula : Int
fechaNacimiento : Date

calcularNotaFinal()
obtenerNombre()

1. **Nombre de la Clase:**
Identifica al objeto.

2. **Atributos:** La información (estado) que la clase almacena.

3. **Operaciones / Métodos:**
El comportamiento que la clase ejecuta.

Una clase encapsula datos y comportamientos relevantes de un objeto de forma sistemática.

Inter Tight: El Espectro de Relaciones

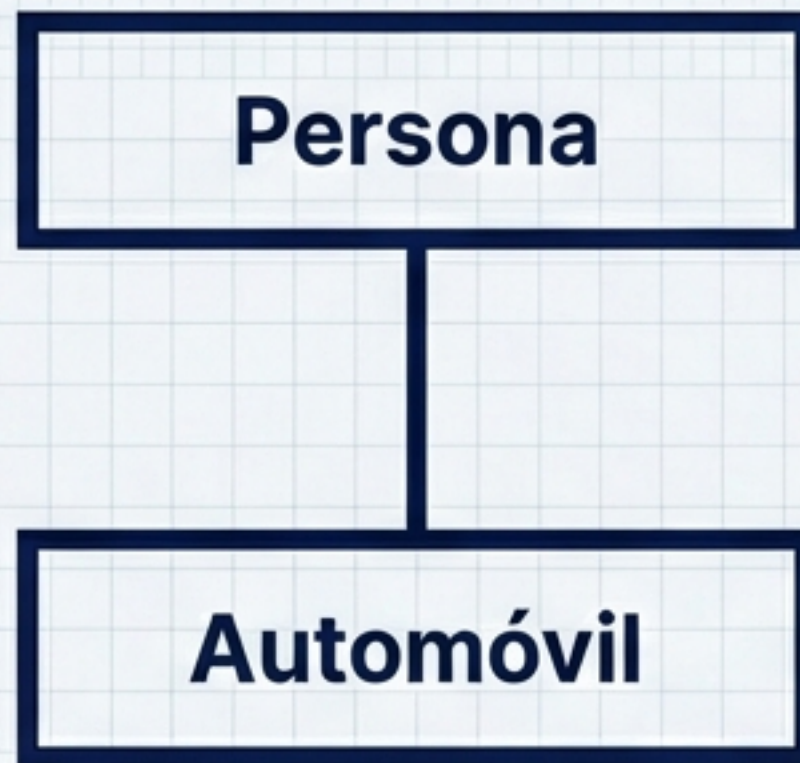
Jerarquía de Acoplamiento (De Débil a Fuerte)



Asociación: La Conexión Básica

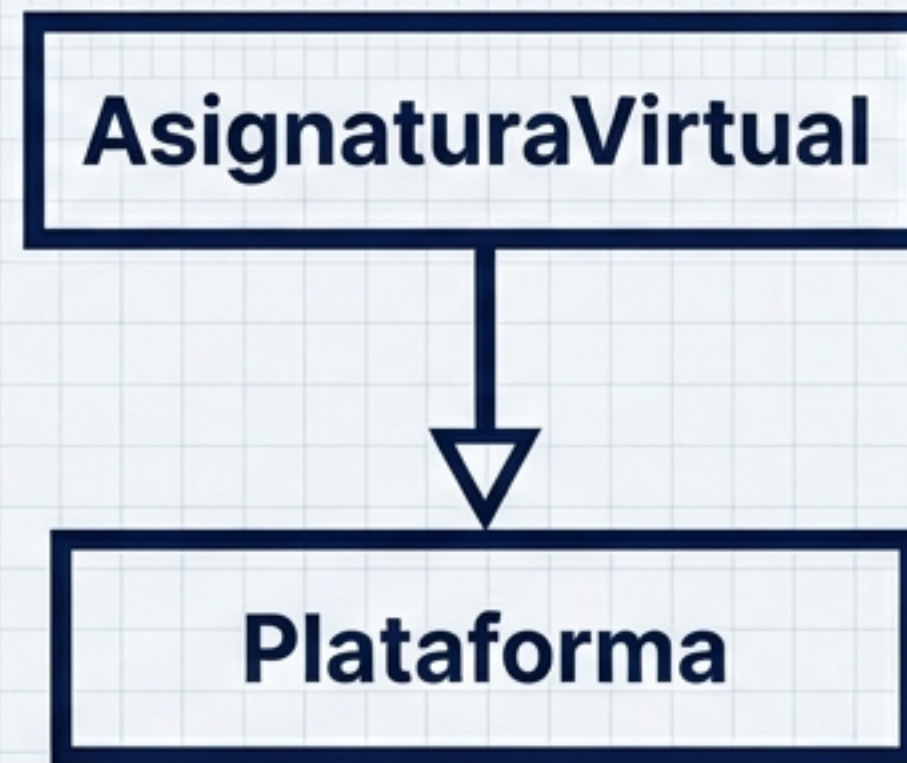
Representa una conexión estructural entre objetos. Puede ser bidireccional (ambos se conocen) o unidireccional.

Bidireccional



Una persona tiene un automóvil y el automóvil pertenece a una persona.

Unidireccional

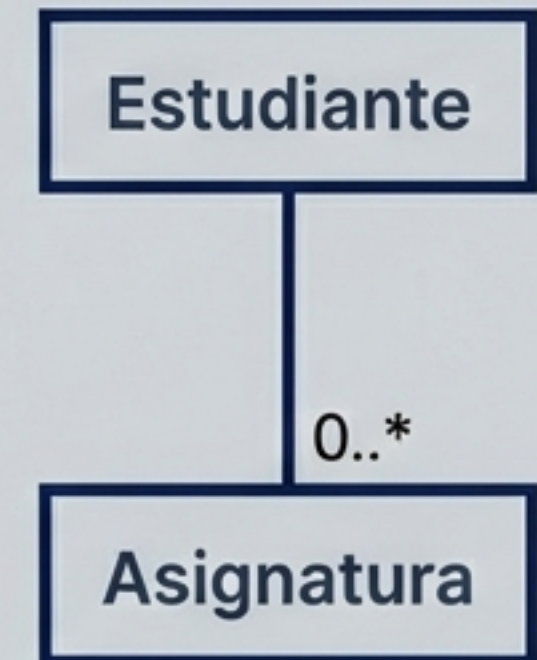


La asignatura *usa* la plataforma, pero la plataforma no conoce a la asignatura.

Multiplicidad

0..* : Cero a muchos

1..1 : Exactamente uno



Agregación: La Colección

Representa una relación débil "Todo-Parte".



Un grupo contiene estudiantes.

Concepto Clave:

Una relación "Todo-Parte" débil. El objeto hijo puede existir independientemente.

Metáfora: Biblioteca y Libros.

Test de Supervivencia

Si el contenedor (Grupo) desaparece, ¿el contenido (Estudiante) sobrevive?

SÍ

El estudiante sigue existiendo en el sistema aunque el grupo se disuelva.

Composición: La Anatomía

Representa una relación fuerte “Todo-Parte”. El ciclo de vida de la parte depende estrictamente del todo.



La asignatura *contiene* temas, que no existen sin ella.

Concepto Clave:

Una relación “Todo-Parte” fuerte. El ciclo de vida de la parte depende estrictamente del todo.

Metáfora: Libro y Páginas.

Test de Supervivencia

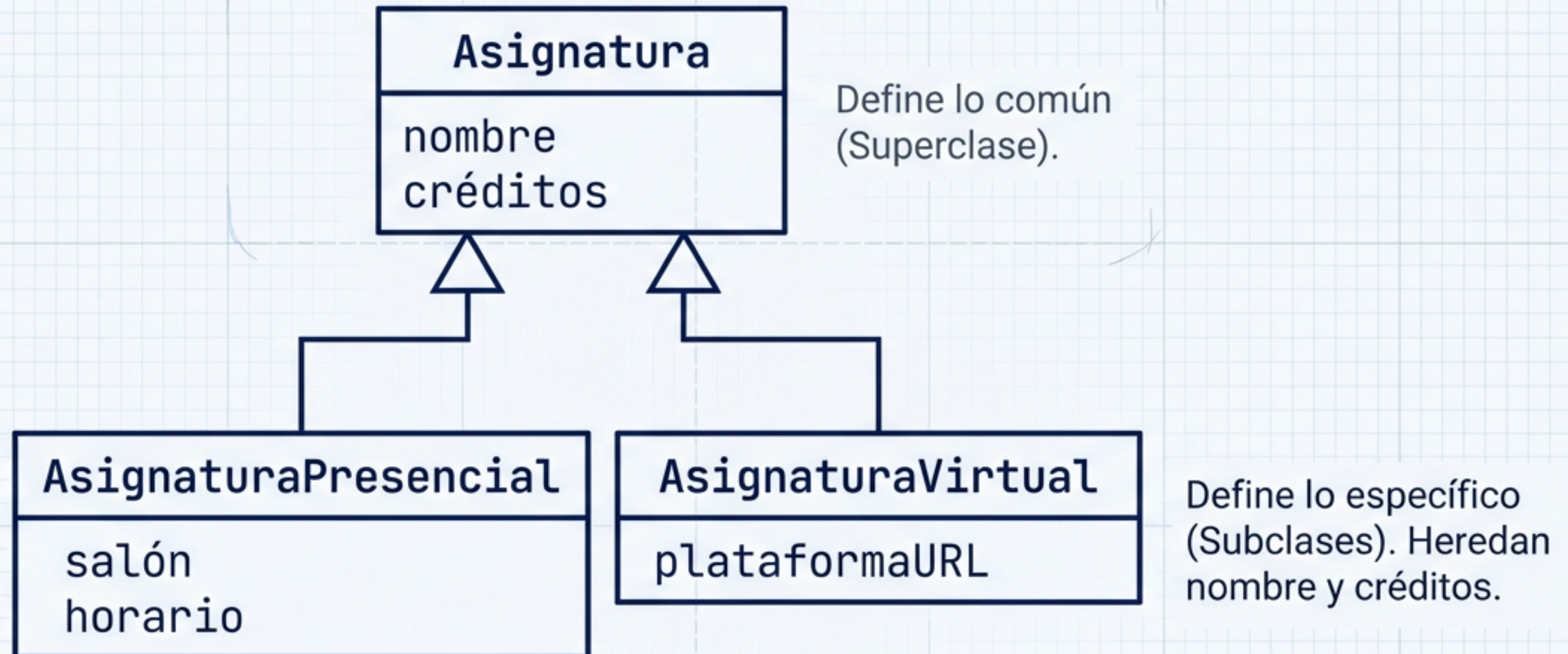
Si el contenedor (Asignatura) desaparece, ¿el contenido (Tema) sobrevive?

NO

Si la asignatura se elimina, los temas pierden sentido y se destruyen.

Generalización: La Herencia

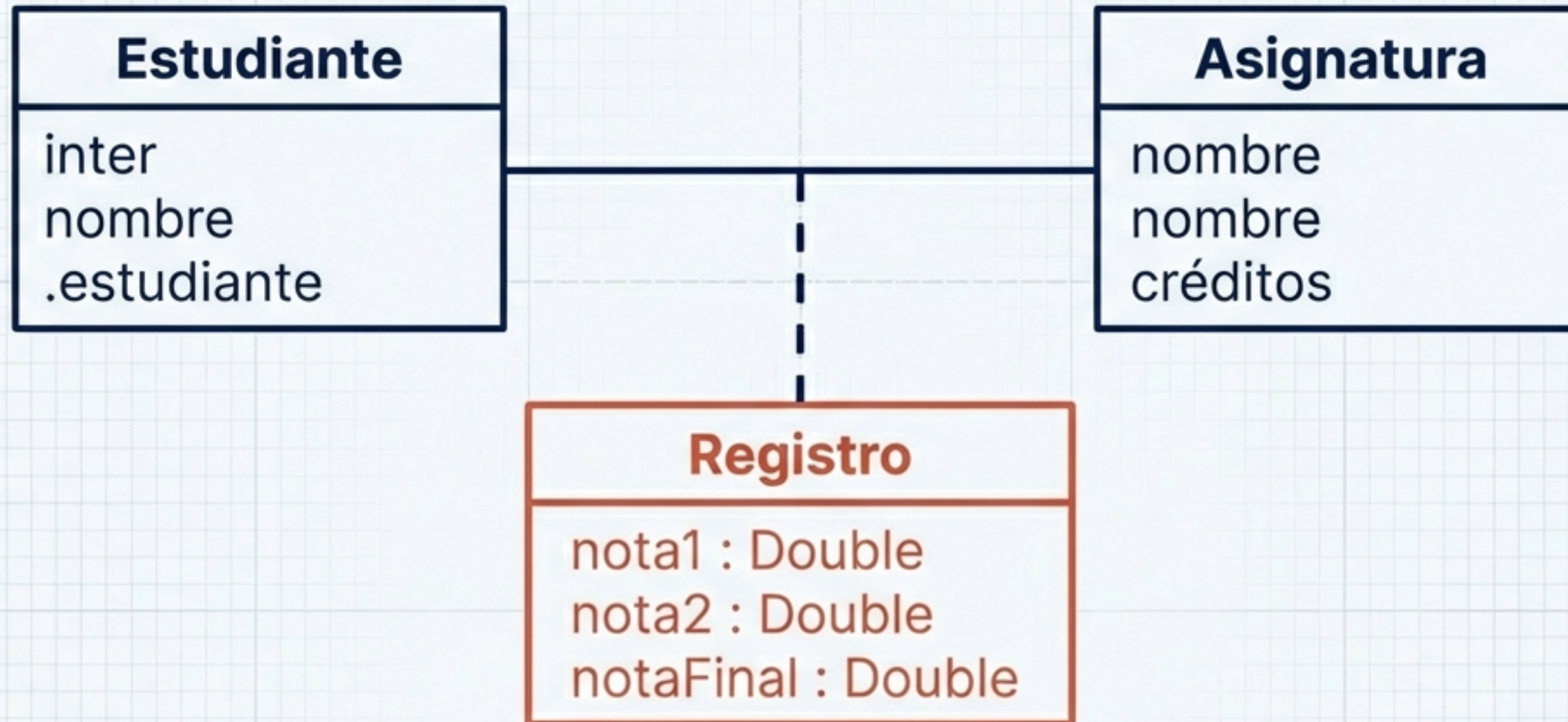
Es un Tipo De...



Clase de Asociación

Cuando el vínculo contiene datos

Un Estudiante tiene una Nota.
Pero, ¿a qué asignatura pertenece esa nota? La nota pertenece al **enlace**.



La clase 'Registro' resuelve la ambigüedad de dónde almacenar las calificaciones.

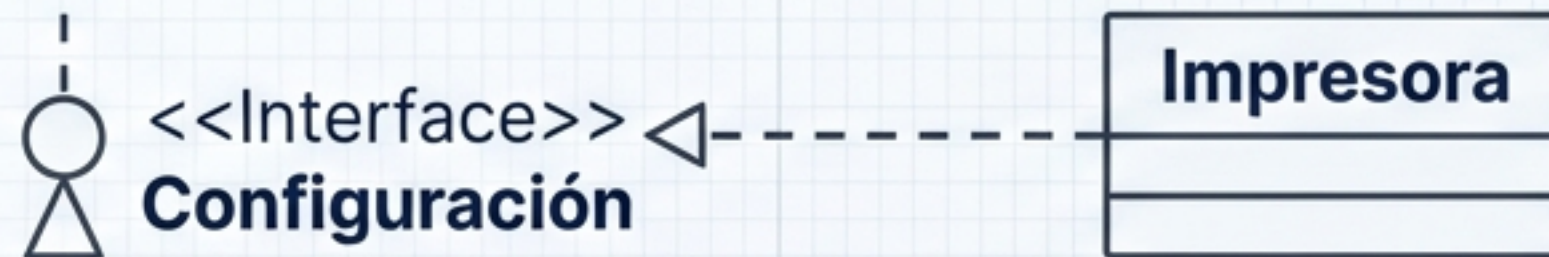
Dependencia y Realización

Dependencia ('Usa')



Un cambio en 'Gasolina' afecta a 'Automóvil'. Relación temporal, usualmente vía parámetros de método.

Realización ('Implementa')



Contrato de comportamiento. La clase 'Impresora' promete cumplir con las funciones definidas en la interfaz.

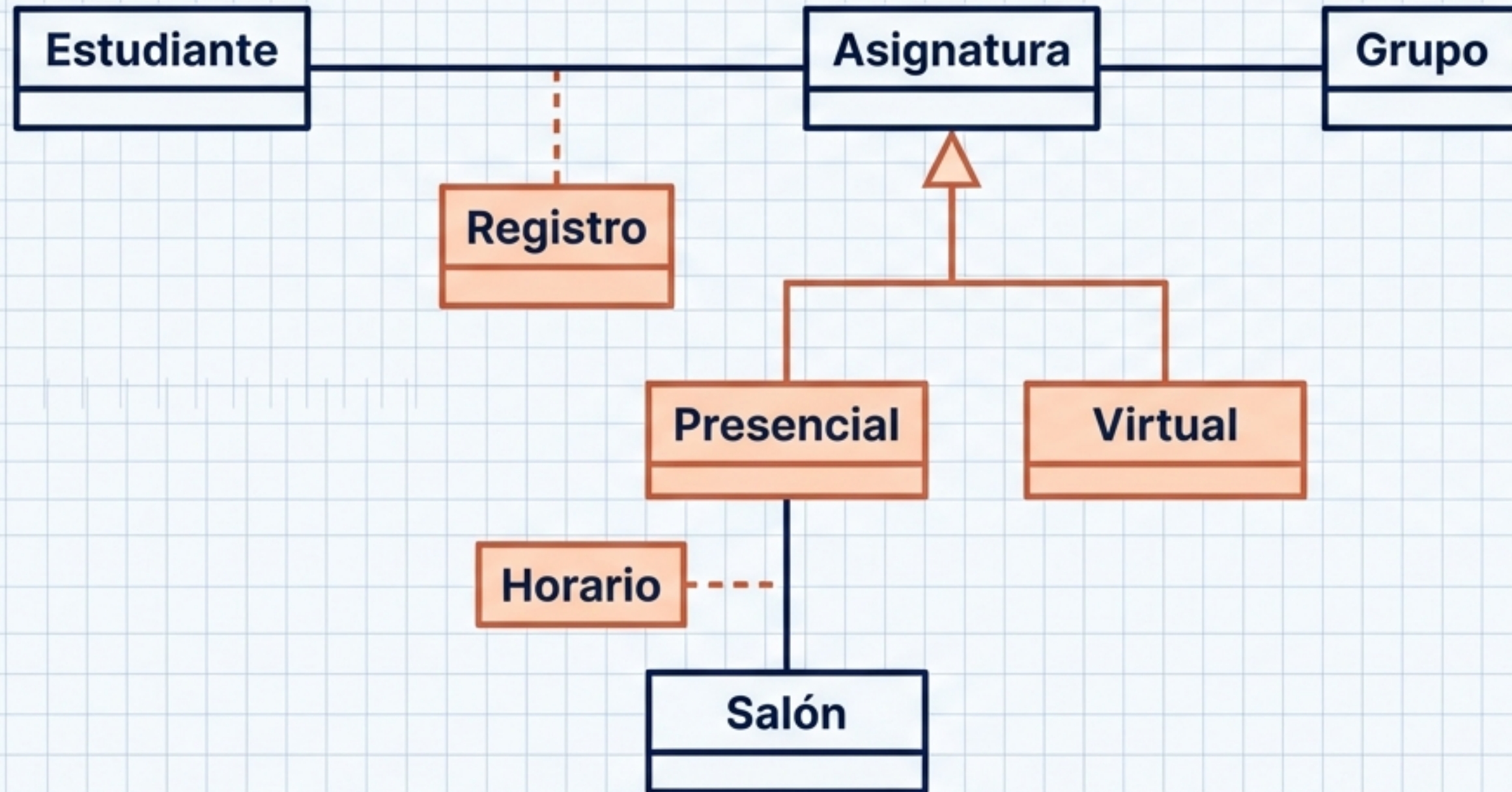
Fase 1: El Modelo de Dominio (Conceptual)

Objetivo: Representar el mundo real.



Fase 2: El Modelo de Análisis (Refinamiento)

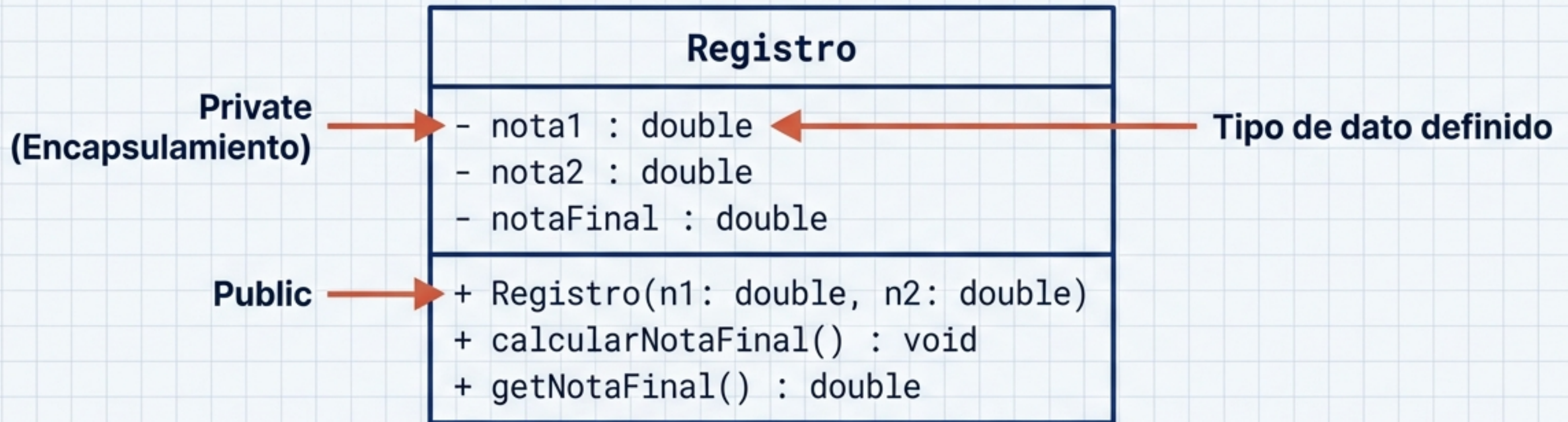
Objetivo: Resolver conflictos lógicos.



Transformación Clave:
'Horario' deja de ser
un simple atributo y se
convierte en una clase
compleja (día, hora
inicio, hora fin) que
vincula Salón y
Asignatura.

Fase 3: El Modelo de Diseño (Listo para Código)

Objetivo: Especificación técnica completa.



Traducción: Del Diagrama al Código (Java)



```
public class Persona {
    private String nombre;
    private String cedula;

    // La asociación se convierte en atributo
    private Automovil auto;

    public void asignarAutomovil(Automovil nuevoAuto) {
        this.auto = nuevoAuto;
    }
}
```

La asociación visual se materializa como un atributo de referencia dentro de la clase.

Conclusión: El Diagrama como Herramienta Viva



Claridad



Evolución



Comunicación

Claridad: Transforma requerimientos abstractos en una estructura tangible.

Evolución: El diagrama crece con el proyecto (Dominio -> Análisis -> Diseño).

Comunicación: Es el lenguaje común entre el experto del negocio y el programador.

Un buen diseño de clases hoy previene la refactorización costosa mañana.